

ML Algorithms Reference Guide

Purpose, when to use, layer/stage, input-output flow, why chosen, and Java code for each algorithm

1. Logistic Regression

Question	Answer
What it accomplishes	Draws ONE smooth boundary and gives a PROBABILITY of belonging to a class - answers 'yes/no, how confident?' (binary classification).
Real-world example	Dependent variable: Will it rain tomorrow (Yes/No). Independent variables: current humidity, pressure, wind speed. Equation: $P(\text{Rain}) = \text{Sigmoid}(w_1 * \text{Humidity} + w_2 * \text{Pressure} + w_3 * \text{Wind} + b)$. Result achieved: a probability like '78% chance of rain' to decide whether to issue an alert.
When used	Binary classification needing a calibrated probability, roughly linear relationship (spam yes/no, credit default yes/no).
Which layer / stage	Whole model in one stage: weighted sum of inputs -> Sigmoid.
Input / Output	In: numeric features. Out: probability 0-1, trained by minimizing Log Loss.
Why select it	Interpretable weights, works well on small/medium linear-ish data.
Splits data or output?	Neither - fits one continuous boundary across all data.
DL or classical?	Classical.

Simple Python example

```
from sklearn.linear_model import LogisticRegression

# x = feature matrix (rows of numeric features), y = labels (0/1)
model = LogisticRegression()          # create the model
model.fit(x, y)                       # train weights by minimizing Log Loss
prediction = model.predict([x[0]])     # predicted class (0 or 1) for row 0
probability = model.predict_proba([x[0]]) # [P(class 0), P(class 1)]
print("Prediction:", prediction, "Probability of class 1:", probability[0][1])
```

2. Support Vector Machine (SVM)

Question	Answer
What it accomplishes	Finds the MAXIMUM-MARGIN boundary between two classes - the most robust possible separating line, not a probability.
Real-world example	Dependent variable: Cyclone likely to make landfall (Yes/No). Independent variables: sea surface temperature, wind shear, pressure drop. Result achieved: a robust yes/no landfall call that stays reliable even with noisy borderline readings.
When used	Binary classification where a clean, robust boundary matters more than confidence score (loan approve/reject).
Which layer / stage	Single stage: computes raw score from support vectors (closest points to the boundary).
Input / Output	In: scaled numeric features. Out: raw score -> side of boundary -> class label.
Why select it	Robust to outliers, works well when classes are cleanly separable.
Splits data or output?	Neither - draws ONE boundary maximizing margin.
DL or classical?	Classical.

Simple Python example

```
from sklearn.svm import SVC

# x = scaled feature matrix, y = labels (0/1)
svm = SVC(kernel="linear") # linear kernel finds a straight max-margin boundary
svm.fit(x, y) # train by minimizing Hinge Loss
prediction = svm.predict([x[0]]) # predicted class based on which side of the margin
print("SVM Prediction:", prediction)
```

3. Decision Tree

Question	Answer
What it accomplishes	Repeatedly SPLITS THE DATA using if-else rules on feature thresholds (e.g. age>20?) until each group is mostly one class.
Real-world example	Dependent variable: Rain today (Yes/No). Rule discovered: IF Soil moisture > 60% AND Cyclone nearby -> Rain. ELSE IF Humidity > 80% -> Rain. ELSE -> No rain. Result achieved: an explainable rulebook a meteorologist could read directly.
When used	Tabular data with nested feature interactions, when you need an interpretable set of rules (Titanic survival).
Which layer / stage	The whole model - no weights, just recursive splitting of rows.
Input / Output	In: raw features, no scaling needed. Out: class label at each leaf (majority vote of rows that land there).
Why select it	Easy to explain (read the splits like a flowchart); captures feature interactions automatically.
Splits data or output?	Splits the DATA (rows) at every node - this is its core mechanic.
DL or classical?	Classical.

Simple Python example

```
from sklearn.tree import DecisionTreeClassifier

# x = raw feature matrix (no scaling needed), y = labels
tree = DecisionTreeClassifier(max_depth=4) # limit depth to avoid overfitting
tree.fit(x, y) # recursively splits data on best feature/threshold
prediction = tree.predict([x[0]]) # predicted class for row 0
print("Tree depth:", tree.get_depth(), "Prediction:", prediction)
```

4. Random Forest

Question	Answer
What it accomplishes	Builds MANY decision trees on random subsets of rows/features, then lets them VOTE - reduces the instability of one single tree.
Real-world example	Dependent variable: Rainfall category (Low/Medium/High) using past 2-year temperature trends, current weather, soil moisture, and cyclone activity as independent variables. Result achieved: a more stable rainfall category prediction than any single rule-based tree, since it averages many trees' opinions.
When used	Tabular data where a single tree overfits (churn, fraud, credit risk).
Which layer / stage	Ensemble stage of many trees trained via bagging (random sampling with replacement).
Input / Output	In: same features as a single tree. Out: majority vote (or averaged probability) across all trees.
Why select it	Much more stable/accurate than one tree, still fairly interpretable via feature importance.

Splits data or output?	Splits DATA at each tree's nodes, and also samples the DATASET across trees (bagging).
DL or classical?	Classical (ensemble).

Simple Python example

```
from sklearn.ensemble import RandomForestClassifier

# x = feature matrix, y = labels
forest = RandomForestClassifier(n_estimators=100) # 100 trees, each on a random row/feature subset
forest.fit(x, y) # trains all trees (bagging)
prediction = forest.predict([x[0]]) # majority vote across all trees
print("Trees:", len(forest.estimators_), "Prediction:", prediction)
```

5. AdaBoost

Question	Answer
What it accomplishes	Trains trees SEQUENTIALLY, each new tree focusing extra weight on the rows the previous tree got WRONG - turns many weak trees into one strong model.
Real-world example	Dependent variable: Flash flood (Yes/No). The first tree misses rare flash-flood cases during low rainfall but high soil saturation; AdaBoost reweights those missed rows so the next tree specifically learns that pattern. Result achieved: better detection of rare, easy-to-miss flood events.
When used	When a single/simple tree underfits and you want iterative improvement (fraud detection).
Which layer / stage	Sequential ensemble: train tree -> reweight misclassified rows -> train next tree.
Input / Output	In: features + evolving per-row weights. Out: weighted vote across all sequential trees.
Why select it	Good when you want to iteratively focus on the hardest examples.
Splits data or output?	Splits DATA at each node, and re-weights rows between rounds.
DL or classical?	Classical (boosting).

Simple Python example

```
from sklearn.ensemble import AdaBoostClassifier

# x = feature matrix, y = labels
ada = AdaBoostClassifier(n_estimators=50) # 50 sequential weak trees
ada.fit(x, y) # each round reweights rows the previous tree got wrong
prediction = ada.predict([x[0]]) # weighted vote across all boosting rounds
print("Rounds:", ada.n_estimators, "Prediction:", prediction)
```

6. Gradient Descent (Optimization Algorithm)

Question	Answer
What it accomplishes	NOT a classifier - it's the engine that ADJUSTS WEIGHTS step by step to minimize a loss function (used to train Logistic Regression, SVM, Neural Nets).
Real-world example	In a rainfall prediction model, Gradient Descent is what tunes the weights w1(temp), w2(weather), w3(soil), w4(cyclone) round after round so the equation's predicted rainfall gets closer to actual recorded rainfall. Result achieved: the actual trained weights, not a prediction itself.
When used	Any time a model has weights to learn and there's no simple closed-form solution.
Which layer / stage	Training stage, runs after the loss is computed each round.

Input / Output	In: current loss + its gradient (slope) w.r.t. each weight. Out: updated weights, fed back for the next round.
Why select it	Standard, scalable way to minimize error over many iterations/epochs.
Splits data or output?	Mini-batch variant splits the DATASET into small batches for faster updates.
DL or classical?	Both - classical model training AND Deep Learning backpropagation.

Simple Python example

```
# Manual gradient descent for a single weight (illustrates the core update rule)
weight = 0.0          # starting weight (randomly initialized in practice)
learning_rate = 0.01  # how big each step is

for epoch in range(100):
    predicted = weight * x[0][0]          # repeat for many iterations
    error = predicted - y[0]              # simple linear prediction using feature 0
    gradient = 2 * error * x[0][0]        # how wrong the prediction was
    weight = weight - learning_rate * gradient # slope of the squared-error loss
                                           # step the weight toward lower loss

print("Learned weight:", weight)
```

7. Gradient Boosting / XGBoost

Question	Answer
What it accomplishes	Trains trees sequentially like AdaBoost, but each new tree predicts the RESIDUAL ERROR left by previous trees - generally the most accurate on tabular data.
Real-world example	Dependent variable: exact rainfall in mm, using past 2-year temperature, current weather, soil moisture, cyclone activity as independent variables. Each new tree corrects the leftover error from prior trees. Result achieved: the most accurate rainfall-in-mm forecast among tree-based methods.
When used	Production tabular classification/regression, competitions, when accuracy matters most.
Which layer / stage	Sequential ensemble: each tree targets the gradient of the loss (residual), summed together.
Input / Output	In: features + current residuals. Out: sum of all trees -> Sigmoid/Softmax for final class.
Why select it	Highest accuracy on structured/tabular data when you can afford more tuning.
Splits data or output?	Splits DATA at each node; targets the OUTPUT ERROR (residual) each round.
DL or classical?	Classical (tree ensemble/boosting).

Simple Python example

```
import xgboost as xgb

# x = feature matrix, y = labels
gbm = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1)
gbm.fit(x, y)          # each new tree fits the previous residual error
prediction = gbm.predict([x[0]]) # sum of all trees -> Sigmoid/Softmax -> final class
print("Boosting rounds:", gbm.n_estimators, "Prediction:", prediction)
```

8. Sigmoid Function + Log Loss

Question	Answer
What it accomplishes	Converts a raw score into a 0-1 PROBABILITY for BINARY classification (2 classes only); Log Loss penalizes confident WRONG predictions heavily.

Real-world example	Raw score from weighted rain features (temp, soil, cyclone) = 2.3 -> Sigmoid turns it into 'probability of rain = 0.91'. Result achieved: a usable percentage chance of rain instead of an abstract score.
When used	Output layer for 2-class problems - Logistic Regression, or the last layer of a binary-output neural network.
Which layer / stage	Output layer / final stage only.
Input / Output	In: raw score (logit). Out: probability, compared to true label via Log Loss to compute error.
Why select it	Whenever there are exactly 2 mutually exclusive classes.
Splits data or output?	Neither - transforms the OUTPUT SCORE of one row.
DL or classical?	Both.

Simple Python example

```
import math

# Sigmoid activation and Log Loss, computed manually
raw_score = 2.3 # weighted sum of inputs (logit)
probability = 1 / (1 + math.exp(-raw_score)) # Sigmoid squashes score into 0-1
true_label = 1 # actual class (0 or 1)
log_loss = -(true_label * math.log(probability) # penalizes wrong-direction confidence
             + (1 - true_label) * math.log(1 - probability))
print("Probability:", probability, "Log Loss:", log_loss)
```

9. Softmax Function + Cross-Entropy Loss

Question	Answer
What it accomplishes	Converts a vector of raw scores into a PROBABILITY DISTRIBUTION across 3+ classes (they sum to 1) - generalizes Sigmoid for MULTI-CLASS classification.
Real-world example	Dependent variable: Weather category (Sunny/Rainy/Cyclonic) instead of just rain yes/no. Independent variables: temp, soil, cyclone signals. Result achieved: probabilities like Sunny=10%, Rainy=25%, Cyclonic=65%, letting you pick the most likely category.
When used	Output layer for 3+ class problems - multinomial logistic regression, or last layer of a multi-class neural network.
Which layer / stage	Output layer / final stage only.
Input / Output	In: vector of raw scores, one per class. Out: probability per class, compared via Cross-Entropy.
Why select it	Whenever there are more than 2 mutually exclusive classes.
Splits data or output?	Neither - transforms the OUTPUT SCORE VECTOR of one row.
DL or classical?	Both.

Simple Python example

```
import math

# Softmax activation and Cross-Entropy, computed manually for 3 classes
raw_scores = [2.0, 1.0, 0.1] # one raw score per class (cat, dog, bird)
sum_exp = sum(math.exp(s) for s in raw_scores) # denominator: sum of all exponentials
probabilities = [math.exp(s) / sum_exp for s in raw_scores] # Softmax: each class's share of total
true_class = 0 # true label index (cat = 0)
cross_entropy = -math.log(probabilities[true_class]) # penalize low probability on true class
print("Probabilities:", probabilities, "Loss:", cross_entropy)
```

10. Hinge Loss (used by SVM)

Question	Answer
What it accomplishes	Penalizes points on the WRONG side of the boundary, or correct but TOO CLOSE to it (inside the margin) - zero penalty once safely correct. Creates SVM's max-margin behavior.
Real-world example	In the cyclone-landfall SVM, a borderline reading (pressure drop close to the threshold) gets penalized even if technically on the correct side, pushing the boundary to stay safely clear of it. Result achieved: a landfall boundary that isn't fooled by near-threshold noise.
When used	Training SVM models.
Which layer / stage	Training/loss-calculation stage, after the raw SVM score is computed.
Input / Output	In: raw SVM score + true label. Out: penalty value used to adjust the boundary.
Why select it	Unlike Log Loss, gives ZERO penalty to confidently-correct points far from the boundary.
Splits data or output?	Neither - scores how well the OUTPUT matches the true label.
DL or classical?	Classical (paired with SVM).

Simple Python example

```
# Hinge Loss computed manually for one data point
raw_score = 0.8          # raw SVM score for this point
true_label = 1           # true label, must be -1 or +1 for hinge loss
hinge_loss = max(0, 1 - true_label * raw_score) # 0 if confidently correct, grows otherwise
print("Hinge Loss:", hinge_loss)
```

11. Linear Regression

Question	Answer
What it accomplishes	Fits ONE straight line/plane through the data to predict a CONTINUOUS NUMBER - not classification, this is regression.
Real-world example	Dependent variable: Rainfall in mm. Independent variables: past 2-year avg temperature, current weather index, soil moisture %, cyclone proximity. Equation: $\text{Rainfall} = w_1 \cdot \text{Temp}_{2\text{yr}} + w_2 \cdot \text{Weather} + w_3 \cdot \text{Soil} + w_4 \cdot \text{Cyclone} + b$. Result achieved: a direct numeric rainfall forecast.
When used	Predicting a number: house price, salary, temperature.
Which layer / stage	Single stage: weighted sum of features, no activation function.
Input / Output	In: numeric features. Out: continuous number, trained by minimizing Mean Squared Error.
Why select it	Simplest baseline when the target is roughly a linear function of features.
Splits data or output?	Neither - fits one continuous line through all data.
DL or classical?	Classical.

Simple Python example

```
from sklearn.linear_model import LinearRegression

# x = feature matrix, y = continuous target values (e.g. house prices)
model = LinearRegression()
model.fit(x, y) # fits weights by minimizing Mean Squared Error
predicted_price = model.predict([x[0]]) # continuous number prediction
print("Predicted value:", predicted_price)
```

12. Ridge & Lasso Regression

Question	Answer
What it accomplishes	Same as Linear Regression, but ADDS A PENALTY on large weights - Ridge shrinks all weights, Lasso can zero out useless ones entirely (automatic feature selection).
Real-world example	Same rainfall equation as Linear Regression, but with 50 correlated weather-station features - Lasso automatically zeroes out redundant stations, keeping only the ones that truly matter. Result achieved: a simpler, less overfit rainfall model.
When used	Many features, risk of overfitting or correlated (multicollinear) features.
Which layer / stage	Same weighted sum, but the loss function adds an L2 (Ridge) or L1 (Lasso) penalty term.
Input / Output	In: numeric features. Out: continuous prediction; some weights shrink or hit zero.
Why select it	Ridge to reduce overfitting; Lasso when you also want automatic feature selection.
Splits data or output?	Neither - still one continuous fit with constrained weight size.
DL or classical?	Classical (same penalty idea = weight decay in Deep Learning).

Simple Python example

```
from sklearn.linear_model import Ridge, Lasso

# x = feature matrix, y = continuous target values
ridge = Ridge(alpha=0.5)    # alpha = penalty strength (higher = more shrinkage)
ridge.fit(x, y)             # L2 penalty shrinks all weights toward zero
lasso = Lasso(alpha=0.5)    # L1 penalty can push some weights to exactly zero
lasso.fit(x, y)
print("Ridge prediction:", ridge.predict([x[0]]))
print("Lasso prediction:", lasso.predict([x[0]]))
```

13. Naive Bayes

Question	Answer
What it accomplishes	Computes the PROBABILITY of each class directly from feature frequencies using Bayes' Theorem, assuming features are independent - fast probabilistic classification.
Real-world example	Dependent variable: Spam or not spam. Independent variables: word frequencies in the email. Result achieved: near-instant spam probability using only how often certain words appear historically in spam vs. non-spam.
When used	Text/categorical classification with limited data: spam filtering, sentiment analysis.
Which layer / stage	Single-stage probabilistic model, no iterative training needed.
Input / Output	In: feature counts/frequencies (e.g. word occurrences). Out: probability per class, picks the highest.
Why select it	Extremely fast to train, works well even with limited data.
Splits data or output?	Neither - computes probabilities directly from statistics.
DL or classical?	Classical (probabilistic).

Simple Python example

```
from sklearn.naive_bayes import MultinomialNB

# x = feature counts (e.g. word frequencies), y = labels
nb = MultinomialNB()
nb.fit(x, y)                # computes class probabilities from feature frequencies
prediction = nb.predict([x[0]]) # predicted class based on highest posterior probability
```

```
print("Naive Bayes Prediction:", prediction)
```

14. K-Nearest Neighbors (KNN)

Question	Answer
What it accomplishes	Predicts by looking at the K CLOSEST existing data points and voting/averaging their labels - no training/boundary formula at all ('lazy learner').
Real-world example	Dependent variable: Rain today (Yes/No). Find the 5 historical days with the most similar temp/soil/cyclone readings to today, and predict rain if most of those 5 days had rain. Result achieved: a prediction based on real similar past days, no formula needed.
When used	Similar inputs should have similar outputs: product recommendations, simple image classification.
Which layer / stage	No training stage - all work happens at prediction time (distance search).
Input / Output	In: scaled numeric features. Out: majority class among K nearest neighbors.
Why select it	Simple, works well when the boundary is highly irregular/non-linear.
Splits data or output?	Neither - searches existing data for neighbors, no splitting.
DL or classical?	Classical (instance-based).

Simple Python example

```
from sklearn.neighbors import KNeighborsClassifier

# x = scaled feature matrix, y = labels
knn = KNeighborsClassifier(n_neighbors=5) # k=5 nearest neighbors
knn.fit(x, y) # just stores the data, no real "training"
prediction = knn.predict([x[0]]) # majority vote among 5 nearest neighbors
print("KNN Prediction:", prediction)
```

15. K-Means Clustering (Unsupervised)

Question	Answer
What it accomplishes	GROUPS similar data points into K clusters with NO LABELS at all - discovers hidden structure instead of predicting a known outcome.
Real-world example	No 'rain' label at all - just group past days by temp/soil/cyclone readings into K=3 natural weather regimes (e.g. 'dry-stable', 'humid-buildup', 'storm-active'). Result achieved: newly discovered weather regime groups useful for further analysis.
When used	No labels available, want to find natural groupings: customer segmentation, grouping similar users.
Which layer / stage	Iterative: assign points to nearest center -> recompute centers -> repeat until stable.
Input / Output	In: numeric features, no labels. Out: a cluster ID per data point.
Why select it	Goal is discovering groups, not predicting a known label.
Splits data or output?	Splits the DATA into K groups - this is its entire purpose.
DL or classical?	Classical (unsupervised).

Simple Python example

```
from sklearn.cluster import KMeans

# x = numeric feature matrix, no labels needed
```



```
kmeans = KMeans(n_clusters=3)      # k=3 groups to find
kmeans.fit(x)                      # assigns points to nearest center, recomputes centers, repeats
cluster_assignments = kmeans.labels_ # cluster ID (0,1,2) assigned to each row
print("Cluster for row 0:", cluster_assignments[0])
```

16. Hierarchical Clustering (Unsupervised)

Question	Answer
What it accomplishes	Builds a TREE OF CLUSTERS (dendrogram) by repeatedly merging the closest points/groups - lets you pick the number of clusters AFTER seeing the tree, unlike K-Means.
Real-world example	Group weather stations by similarity in temp/soil/cyclone patterns without deciding K upfront, then cut the tree at different heights to see 3 regimes or 6 finer sub-regimes. Result achieved: flexible, multi-level view of weather groupings.
When used	Don't know K in advance, want to see nested groupings: gene expression, document/topic grouping.
Which layer / stage	Iterative merging: start with every point as its own cluster, merge closest pairs repeatedly.
Input / Output	In: numeric features, no labels. Out: a dendrogram (tree), cut at any height to get K clusters.
Why select it	Chosen when you want to explore cluster structure at multiple levels of granularity.
Splits data or output?	Groups the DATA (opposite direction of Decision Tree - merges instead of splits).
DL or classical?	Classical (unsupervised).

Simple Python example

```
from sklearn.cluster import AgglomerativeClustering

# x = numeric feature matrix, no labels needed
hc = AgglomerativeClustering(n_clusters=3, linkage="complete") # merge closest pairs repeatedly
cluster_assignments = hc.fit_predict(x) # cut the resulting tree to get 3 clusters
print("Cluster for row 0:", cluster_assignments[0])
```

17. DBSCAN (Density-Based Clustering)

Question	Answer
What it accomplishes	Groups points that are DENSELY PACKED together and automatically marks sparse/isolated points as OUTLIERS/NOISE - unlike K-Means, it doesn't force every point into a cluster.
Real-world example	Cluster days with dense, similar cyclone-activity readings together, while automatically flagging a single freak storm reading (unlike anything nearby) as noise/outlier. Result achieved: natural storm-pattern groups plus automatic anomaly flagging.
When used	Clusters of irregular shapes, or when you need automatic outlier/anomaly detection alongside clustering (fraud detection, geographic hotspots).
Which layer / stage	Iterative: expands clusters from 'core points' that have enough neighbors within a radius.
Input / Output	In: numeric features + 2 parameters (radius, min points). Out: cluster ID per point, or -1 for noise/outlier.
Why select it	Chosen when clusters aren't round/even-sized (K-Means assumes round clusters) or outlier detection matters.
Splits data or output?	Groups the DATA based on density, leaving sparse points unclustered.

DL or classical?	Classical (unsupervised).
------------------	---------------------------

Simple Python example

```
from sklearn.cluster import DBSCAN
```

```
# x = numeric feature matrix, no labels needed
dbscan = DBSCAN(eps=0.5, min_samples=5) # eps=radius, min_samples=min points to form a dense region
cluster_assignments = dbscan.fit_predict(x) # cluster ID per row, -1 means noise/outlier
print("Cluster for row 0:", cluster_assignments[0])
```

18. Principal Component Analysis (PCA)

Question	Answer
What it accomplishes	COMPRESSES many correlated features into fewer new features (principal components) that capture most of the original information/variance - a preprocessing step, not a predictor.
Real-world example	50 correlated weather-station readings (temp, humidity, pressure at nearby stations) compressed into 5 principal components that still capture 95% of the pattern, before feeding into a rain prediction model. Result achieved: faster training, less noise.
When used	Too many features (high dimensionality), before feeding data into another model, or to visualize data in 2D/3D.
Which layer / stage	Preprocessing stage, runs BEFORE the actual classifier/regressor.
Input / Output	In: scaled numeric features. Out: fewer transformed features, fed into Logistic Regression/SVM/Neural Net as new input.
Why select it	Speeds up training, reduces overfitting, or enables visualization of high-dimensional data.
Splits data or output?	Neither - transforms/compresses the FEATURE SPACE, not rows or output.
DL or classical?	Classical (linear algebra based; Autoencoders do a similar job in Deep Learning).

Simple Python example

```
from sklearn.decomposition import PCA
```

```
# x = scaled numeric feature matrix
pca = PCA(n_components=2) # keep only the top 2 components
reduced_features = pca.fit_transform(x) # compressed feature matrix, now used for modelling
print("Reduced feature count:", reduced_features.shape[1])
```

19. Linear Discriminant Analysis (LDA)

Question	Answer
What it accomplishes	Finds the axis that BEST SEPARATES known classes while reducing dimensions - unlike PCA (which ignores labels), LDA uses the labels to choose directions that maximize class separation.
Real-world example	Same 50 weather-station features, but this time using the known Rain/No-Rain label to choose the compressed axes that best separate rainy days from dry days, not just the axes with the most variance. Result achieved: fewer features, better tuned for the rain classification task specifically.
When used	Classification + dimensionality reduction together, when you already have labels (face recognition, marketing segments with known outcome).

Which layer / stage	Preprocessing/model stage: projects data onto axes that maximize between-class distance, minimize within-class spread.
Input / Output	In: numeric features + class labels. Out: reduced features that best separate the classes, or a direct class prediction.
Why select it	Chosen over PCA when you have labels and want the reduction to help classification specifically, not just capture variance.
Splits data or output?	Neither - transforms feature space using label information.
DL or classical?	Classical.

Simple Python example

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# x = numeric feature matrix, y = labels (used to choose separating axes)
lda = LinearDiscriminantAnalysis()
lda.fit(x, y) # finds axes that maximize class separation
prediction = lda.predict([x[0]]) # predicted class using label-aware axis separation
print("LDA Prediction:", prediction)
```

20. Isolation Forest (Anomaly Detection)

Question	Answer
What it accomplishes	Finds OUTLIERS/ANOMALIES by randomly splitting data - anomalies get ISOLATED in very few splits (they're rare and different), while normal points take many splits to isolate.
Real-world example	Flag a sensor reading where soil moisture spiked to an impossible value in seconds - it gets isolated in just 1-2 random splits because it's so different from normal readings. Result achieved: automatic detection of faulty sensors or freak weather events, no labels needed.
When used	Fraud detection, network intrusion detection, finding defective sensor readings - no labels needed.
Which layer / stage	Ensemble of random trees, each randomly splitting features/thresholds; anomaly score = average path length to isolate a point.
Input / Output	In: numeric features, no labels. Out: an anomaly score per point (short path = more anomalous).
Why select it	Chosen specifically for anomaly detection - fast, works well in high dimensions, no need for labeled anomalies.
Splits data or output?	Splits the DATA randomly at each tree node (like Decision Tree, but random, not optimized).
DL or classical?	Classical (tree-based ensemble).

Simple Python example

```
from sklearn.ensemble import IsolationForest

# x = numeric feature matrix, no labels needed
iso_forest = IsolationForest()
iso_forest.fit(x) # builds random trees; anomalies isolate faster
anomaly_score = iso_forest.decision_function([x[0]]) # lower score = more anomalous
print("Anomaly score for row 0:", anomaly_score)
```

21. Neural Networks (Deep Learning)

Question	Answer
----------	--------

What it accomplishes	Stacks many weighted layers with non-linear activations to learn COMPLEX, NON-LINEAR patterns directly from raw/high-dimensional data (pixels, text, audio) - no manual feature engineering needed.
Real-world example	Instead of hand-picked features (temp, soil, cyclone), feed raw satellite cloud images directly - the network learns its own complex patterns (cloud shapes, movement) to predict rain. Result achieved: rain prediction from raw imagery, without manually engineering features.
When used	Images, text, audio, video - complex patterns that simpler models (trees, logistic regression) underfit.
Which layer / stage	MULTIPLE stacked layers: weighted sum -> activation (ReLU hidden layers, Sigmoid/Softmax output) -> next layer.
Input / Output	In: raw/lightly processed data. Out: final Sigmoid/Softmax probability, trained via Gradient Descent + Backpropagation.
Why select it	Chosen when data is large, high-dimensional/unstructured, and simpler models underfit the pattern complexity.
Splits data or output?	Neither directly - transforms data through successive weighted layers.
DL or classical?	This IS Deep Learning - 'deep' means multiple hidden layers stacked together.

Simple Python example

```
from sklearn.neural_network import MLPClassifier
```

```
# x = feature matrix (e.g. pixels, embeddings), y = labels
nn = MLPClassifier(hidden_layer_sizes=(16,), activation="relu", max_iter=50)
# 16 hidden neurons (ReLU), output layer auto-uses Sigmoid/Softmax based on class count
nn.fit(x, y) # trains via Gradient Descent + Backpropagation
probability = nn.predict_proba([x[0]]) # predicted probability for row 0
print("NN predicted probability:", probability)
```

22. Cosine Similarity

Question	Answer
What it accomplishes	Measures whether two things point in the SAME DIRECTION (same type/meaning), ignoring their size - used to group or compare items, not to classify into a fixed label.
Apple vs Orange, layman terms	Imagine each fruit described as a vector: [Sweetness, Acidity]. Apple = [4, 2], another Apple = [5, 2.5] (bigger fruit, more juice, but SAME ratio of sweet-to-acid). Orange = [2, 5] (very different ratio - more acidic than sweet).
Why dot product increasing means same group	Dot product = (Sweetness1 x Sweetness2) + (Acidity1 x Acidity2). When two vectors point the SAME direction, their dot product grows large relative to their lengths - because every part of one vector is being multiplied by a matching large part of the other, instead of cancelling out. Apple-vs-Apple: dot product is large AND cosine stays close to 1, because direction matches even though size (juiciness) differs. Apple-vs-Orange: dot product is smaller relative to their lengths, because the ratio (direction) is different, so cosine similarity is low.
Result achieved	Cosine similarity near 1 -> SAME group (both 'Apple-like', regardless of size). Cosine similarity near 0 -> DIFFERENT group (Apple vs Orange), even if their raw magnitudes/dot product number itself looks big.
Which layer / stage	Not a model layer - a comparison metric, often used after embeddings/vectors are produced (e.g. by a Neural Network) to group or retrieve similar items.

Real-world example	Recommendation systems: represent each product as a vector of features (price range, category signals). A cheap and an expensive item in the SAME category still point the same direction (high cosine similarity) and get grouped/recommended together, even though their 'price' magnitude differs a lot.
--------------------	---

Simple Python example

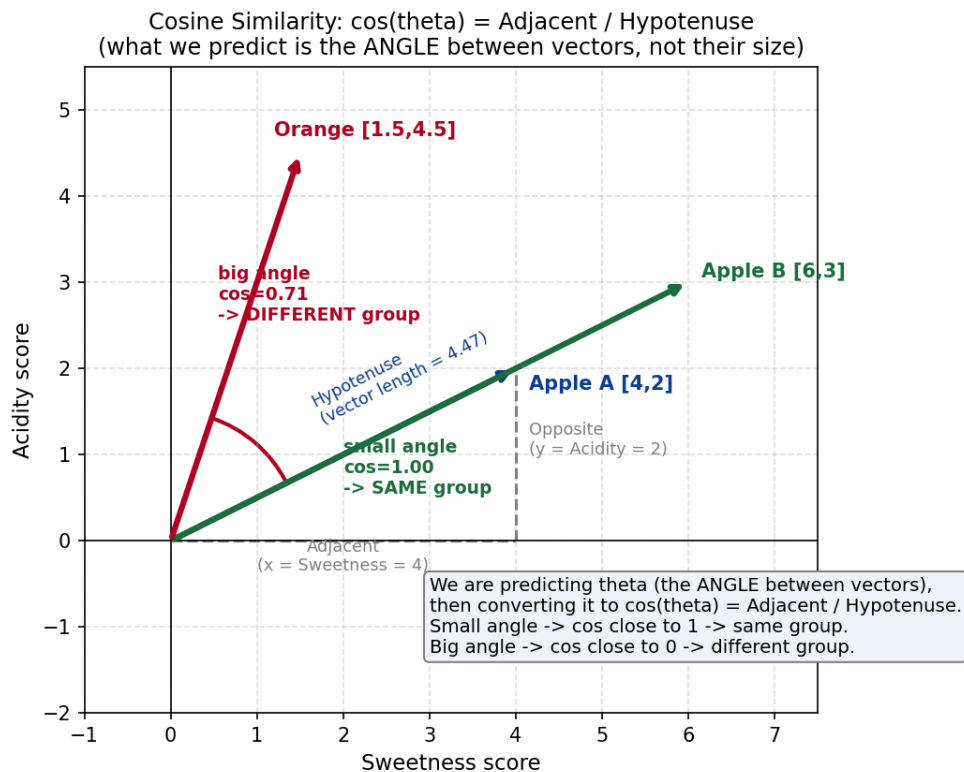
```
import numpy as np

# Two apples with different sizes, but the SAME sweetness-to-acidity ratio (same direction)
apple1 = np.array([4, 2])    # [Sweetness, Acidity]
apple2 = np.array([8, 4])    # bigger apple, exact same ratio -> same direction
orange = np.array([2, 5])    # very different ratio -> different direction

def cosine_similarity(v1, v2):
    dot_product = np.dot(v1, v2)                # how much they "agree" in raw terms
    magnitude = np.linalg.norm(v1) * np.linalg.norm(v2) # strips out size/length
    return dot_product / magnitude              # pure direction match, from -1 to 1

print("Apple vs Apple similarity:", cosine_similarity(apple1, apple2)) # close to 1.0 -> same group
print("Apple vs Orange similarity:", cosine_similarity(apple1, orange)) # much lower -> different group
```

Diagram: $\cos(\theta) = \text{Adjacent} / \text{Hypotenuse}$ - small angle between vectors means same group



23. Sigma (Σ) Notation - What It Means

Question	Answer
----------	--------

What it accomplishes	NOT an algorithm - it's math shorthand for 'ADD UP the same formula, repeated for every data point.' Used constantly inside loss functions (Log Loss, Cross-Entropy, MSE) to combine errors from every row into one number.
How to read it	Bottom of Sigma ($i=1$) = where to start looping. Top of Sigma (n) = last data point to include. The expression to the right = the formula repeated for EACH i , then all results are added together.
Real-world example	Predicting rainfall for 3 days: actual = [3,5,2] mm, predicted = [2.5,5.5,1] mm. Sigma of (actual-predicted)^2 means: compute the squared error for Day 1, Day 2, Day 3 separately, THEN add all 3 together into one total error number.
Why it matters for ML	Every model needs ONE overall error number (not one number per row) to know if it's improving. Sigma is simply how you turn 'many individual errors' into 'one total error' that Gradient Descent can minimize.

Simple Python example

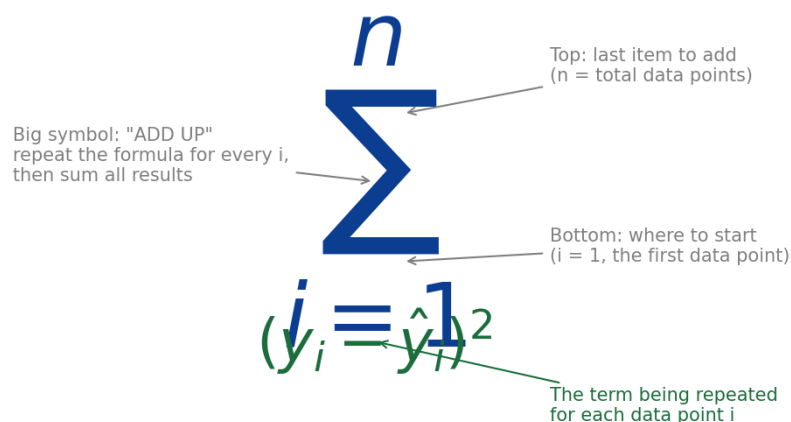
```
# Sigma (sum) in plain code: total error across all data points
y_true = [3, 5, 2]      # actual rainfall for 3 days
y_pred = [2.5, 5.5, 1.0] # predicted rainfall for the same 3 days

total = 0
for i in range(len(y_true)):
    term = (y_true[i] - y_pred[i]) ** 2  # this loop IS the Sigma symbol
    print(f"Term i={i+1}: {term}")       # the formula repeated for each i
    total += term                        # add each term into the running total

print("Sigma result (total error):", total)
```

Diagram: what each part of the Sigma symbol means

Σ (Sigma) = "loop through $i=1$ to n , and add up each result"



24. Why Log Loss Uses $\log()$ - High Penalty for Confident Wrong Predictions

Question	Answer
What it accomplishes	Turns 'how wrong was the prediction' into a penalty that grows EXPLOSIVELY (not just linearly) as the model gets more confidently wrong - this is what makes Log Loss punish bad false positives/negatives so harshly.

The equation	Log Loss for one row = $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$, where y = true label (0 or 1), p = predicted probability. Only ONE of the two terms is active depending on whether $y=0$ or $y=1$.
Why log specifically (not just 1-p)	A straight-line penalty (like $1-p$) grows the SAME amount whether the model is a little wrong or very wrong. $\log(p)$ instead stays small while p is close to the true answer, then shoots up toward infinity as p approaches the WRONG extreme (0) - so a confidently wrong prediction gets punished far more than a borderline one.
Real-world example	Predicting 'no rain' with $p=0.9$ confidence when it actually rains: loss = $-\log(0.1) = 2.30$ (already a big penalty). Predicting the same wrong outcome with $p=0.99$ confidence: loss = $-\log(0.01) = 4.60$ - DOUBLE the penalty for being even more confidently wrong. This is exactly how $\log()$ creates a high penalty specifically for confident false positives/negatives.
Which layer / stage	Training/loss-calculation stage, used after Sigmoid (2 classes) or Softmax (3+ classes) produces a probability.

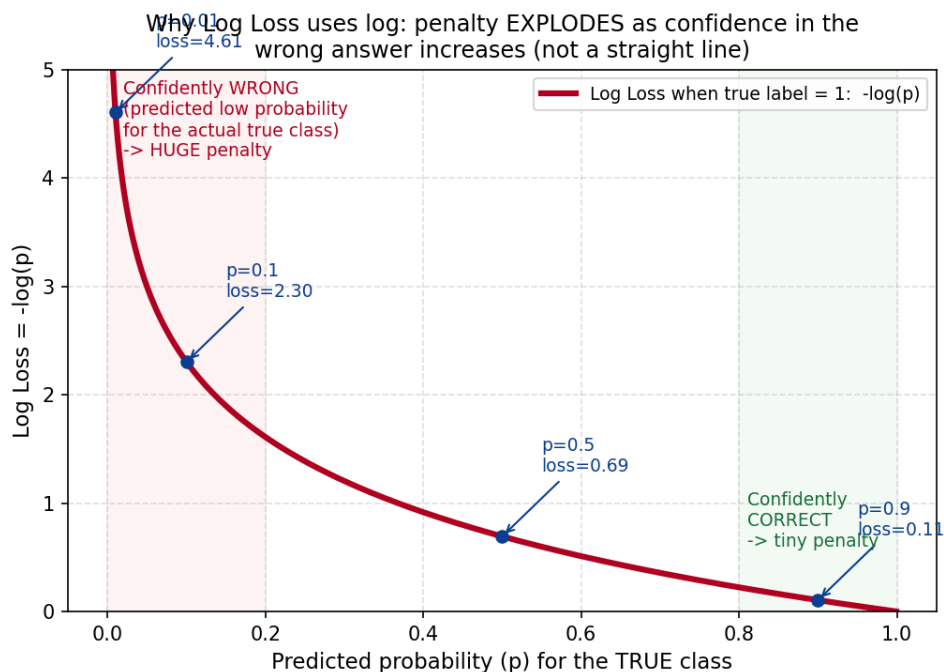
Simple Python example

```
import math

def log_loss_single(y_true, p_predicted):
    # Only one term is active: the y*log(p) part if true label is 1,
    # or the (1-y)*log(1-p) part if true label is 0
    return -(y_true * math.log(p_predicted) + (1 - y_true) * math.log(1 - p_predicted))

# Same WRONG prediction (true label=1), but increasing confidence in the wrong answer
print("Mildly wrong (p=0.4):", log_loss_single(1, 0.4)) # small-ish penalty
print("Wrong (p=0.1):", log_loss_single(1, 0.1)) # bigger penalty
print("Very confidently wrong (p=0.01):", log_loss_single(1, 0.01)) # huge penalty
```

Diagram: Log Loss curve - penalty explodes as confidence in the wrong answer grows



Quick Decision Guide

Predicting a NUMBER → Linear Regression, or Ridge/Lasso if many features.

Tabular data with nested feature interactions → Decision Tree / Random Forest / Gradient Boosting (XGBoost).

Smooth, linear relationship, need probability, small dataset → Logistic Regression.

Need a robust, max-margin boundary, no probability needed → SVM.

Text/categorical data, need something fast with little data → Naive Bayes.

No formula needed, just compare to similar past examples → K-Nearest Neighbors.

No labels, want round/equal-sized groups, know K → K-Means Clustering.

No labels, don't know K, want to see nested groupings → Hierarchical Clustering.

No labels, irregular-shaped clusters, need outlier detection → DBSCAN.

Too many features, want to compress before modelling (no labels used) → PCA.

Too many features, HAVE labels, want reduction to help classification → LDA.

Need to flag rare/fraudulent/abnormal records → Isolation Forest.

High-dimensional raw data (images, text, audio) → Neural Network (Deep Learning) with Sigmoid/Softmax output.

Single tree overfits or is unstable → Random Forest (bagging) or AdaBoost / Gradient Boosting (sequential boosting).